# Real-Time Schedulers for STM32: Performance Evaluation of EDF over SuperLoop

Vladimir Toporkov, Artem Burmyakov
*Innopolis University*

## I. Introduction

STM32 is a family of microcontrollers developed by STMicroelectronics for embedded applications. These devices integrate a processor, memory, and peripheral interfaces on a single chip [1].

STM32-based devices support applications such as environmental monitoring [2] and equipment condition monitoring [3]. In these applications maintaining responsiveness as the workload increases is a key challenge.

The paper examines how the choice of scheduling approach affects responsiveness and processing cost on STM32. We compare two approaches: SuperLoop [4] and Earliest Deadline First (EDF) [5].

### A. Contribution

We compare periodic SuperLoop and chunked EDF on an STM32F767. We evaluate the effects of workload utilization and chunk duration on response time, deadline misses, skipped releases, and measured selection overhead. Our implementations for both schedulers are publicly available.

## II. Background on STM32

Fig. 1 illustrates the sensor platform considered in this work, based on an STM32F767 uniprocessor. Table I summarizes the functions and interfaces of its four sensors.

TABLE I
Sensor functions and interfaces.

| Sensor | Purpose | Interface |
|---|---|---|
| DHT11 [6] | Temperature and humidity | Single-wire digital (GPIO) |
| HC-SR04 [7] | Distance | Trigger/echo (GPIO) |
| OV2640 [8] | Image acquisition | Parallel data, SCCB |
| SPW2430 [9] | Signal acquisition | Analog output (ADC) |

General-purpose input/output (GPIO) pins carry digital signals, while an analog-to-digital converter (ADC) converts an analog signal into digital values. The Serial Camera Control Bus (SCCB) configures the camera.

### A. Programming principles

**1.AB: To describe the key aspects of STM32 programming. Limit the text to one page.**

1) Programming environment: software name, protocols, etc.
2) Template structure of the project (macroses) in the development environment;
3) Functionality (macroses) to be implemented in that structure;
4) Put some print screens.

## III. System Model

We use the periodic task model of Liu and Layland [5] and adapt the task and release notation of Burmyakov and Bini [10]. Table II summarizes the notation.

A real-time system $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$ consists of $n$ periodic tasks executed on one core ($m = 1$). Each task $\tau_i = (C_i, D_i, P_i)$ is defined by the worst-case execution time $C_i$ of a job, a relative deadline $D_i$, and a period $P_i$. We consider implicit deadlines, $D_i = P_i$. Task utilization is $U_i = {}^{C_i}/_{P_i}$, and task-set utilization is $U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$.

Let $J_{i,j}$ denote the $j$-th job of $\tau_i$, released at $r_{i,j}$. Consecutive jobs are released one period apart:

$$r_{i,j+1} = r_{i,j} + P_i. \tag{1}$$

Each job $J_{i,j}$ has an absolute deadline $d_{i,j} = r_{i,j} + D_i$. If completed, its finish time is denoted by $f_{i,j}$.

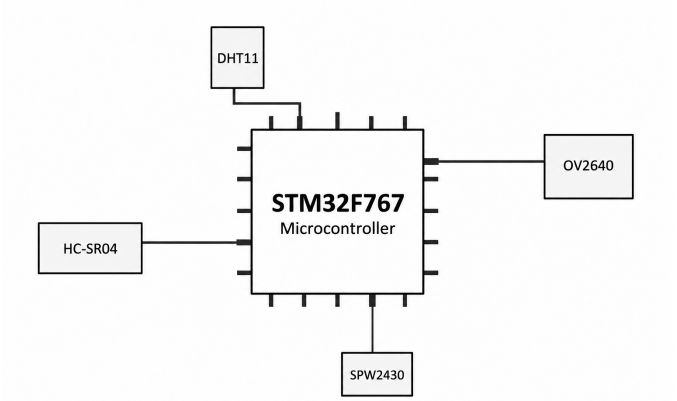



Fig. 1. Sensor platform based on STM32F767.

TABLE II
SYSTEM MODEL NOTATION.

| Symbol | Meaning |
|---|---|
| $\mathcal{T}, n$ | Task set and number of tasks |
| $\tau_i$ | Task $i$ |
| $C_i$ | Worst-case execution time of a job of $\tau_i$ |
| $P_i$ | Task period |
| $D_i = P_i$ | Relative deadline, equal to the period |
| $J_{i,j}$ | The $j$-th job of task $\tau_i$ |
| $r_{i,j}$ | Release time of job $J_{i,j}$ |
| $f_{i,j}$ | Job completion time |
| $d_{i,j} = r_{i,j} + D_i$ | Absolute deadline of job $J_{i,j}$ |
| $U_i = C_i/P_i$ | Task utilization |
| $U_\mathcal{T} = \sum_{\tau_i \in \mathcal{T}} U_i$ | Task-set utilization |

- A schedule specifies which job executes at each time, if any.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job $J_{i,j}$ misses its deadline if $f_{i,j} > d_{i,j}$.
- The response time of a completed job $J_{i,j}$ is $f_{i,j} - r_{i,j}$.

## IV. SCHEDULERS

We consider two schedulers: SuperLoop and EDF implementation in which jobs execute in chunks.

### A. SuperLoop

SuperLoop repeatedly polls tasks in a fixed order. For each task, it checks whether the next release time has been reached. If so, the job runs to completion before the next task is polled. The key steps of the SuperLoop scheduler are outlined in Algorithm 1.

**Algorithm 1** SuperLoop Scheduler

```
function SUPERLOOP(T)
    t_i^next ← 0,  i = 1, ..., n                ▷ Initial releases
    while true do
        for i ← 1 to n do                        ▷ Fixed task order
            t^cur ← NOW()                        ▷ Read current system time
            if t_i^next ≤ t^cur then
                J_{i,j} ← RELEASEJOB(τ_i, t_i^next)
                EXECUTE(J_{i,j})                 ▷ Non-preemptive
                f_{i,j} ← NOW()
                t_i^next ← t_i^next + P_i
                while t_i^next < f_{i,j} do t_i^next ← t_i^next + P_i
            end if
        end for
    end while
end function
```

Fig. 2(a) illustrates the SuperLoop schedule for tasks $\tau_1$ and $\tau_2$. Both tasks release their first jobs at $t = 0$. Since $\tau_1$ comes first in the polling order, $J_{1,1}$ executes until $t = 18$ ms. Job $J_{2,1}$ then starts and completes at $f_{2,1} = 28$ ms, missing its deadline $d_{2,1} = 20$ ms.

The release at $r_{2,2} = 20$ ms occurs while $J_{2,1}$ is still running. Once this job completes, the scheduler counts that release as skipped and advances the next release time to $r_{2,3} = 40$ ms. Polling continues with no ready job until then.

### B. Earliest Deadline First

In contrast, preemptive EDF selects the ready job with the earliest absolute deadline. A newly released job preempts the running job if its deadline is earlier.

Fig. 2(b) shows the EDF schedule for the same task set. At $t = 0$, $J_{2,1}$ has the earlier deadline and executes until $t = 10$ ms. Job $J_{1,1}$ then starts, but is preempted at $r_{2,2} = 20$ ms because the new job has a deadline $d_{2,2} = 40$ ms, earlier than $d_{1,1} = 50$ ms. Job $J_{2,2}$ executes until $t = 30$ ms, after which $J_{1,1}$ resumes and completes at $t = 38$ ms. The next job of $\tau_2$ executes from $t = 40$ to $t = 50$ ms. Thus, no deadline is missed and no release is skipped in this example.

We use chunked EDF to evaluate scheduling on STM32 before developing a fully preemptive implementation. Jobs run in non-preemptive chunks of at most $\Delta$ ms.

Algorithm 2 outlines chunked EDF. The scheduler maintains a set $\mathcal{R}$ of released but unfinished jobs. After each chunk, it updates $\mathcal{R}$ with newly released jobs and selects the job with the earliest absolute deadline.

Fig. 2(c) shows the same task set with $\Delta = 4$ ms. At $r_{2,2} = 20$ ms, the first job of $\tau_1$ is still executing a chunk, which ends at $t = 22$ ms. The new job of $\tau_2$ therefore starts at $t = 22$ ms, with a preemption delay of $\delta = 2$ ms. It executes in chunks of 4, 4, and 2 ms and completes at $t = 32$ ms. The shorter final chunk reflects the remaining work. The interrupted job of $\tau_1$ then resumes and completes at $t = 38$ ms.

**Algorithm 2** Chunked EDF Scheduler

```
function CHUNKEDEDF(T, Δ)
    R ← ∅                                        ▷ Set of unfinished jobs
    t_i^next ← 0,  i = 1, ..., n                ▷ Initial releases
    while true do
        t^cur ← NOW()                            ▷ Read current system time
        for all τ_i ∈ T do                       ▷ Check releases
            if t_i^next ≤ t^cur and no τ_i job in R then
                J_{i,j} ← RELEASEJOB(τ_i, t_i^next)
                R ← R ∪ {J_{i,j}}                ▷ Add new job
            end if
        end for
        if R ≠ ∅ then                            ▷ Pending work
            J_{i,j} ← SELECTEDF(R)               ▷ Earliest deadline
            EXECUTECHUNK(J_{i,j}, Δ)             ▷ Duration ≤ Δ
            if COMPLETED(J_{i,j}) then
                f_{i,j} ← NOW()
                while t_i^next < f_{i,j} do t_i^next ← t_i^next + P_i
                R ← R \ {J_{i,j}}                ▷ Remove completed job
            end if
        end if
    end while
end function
```

$$\tau_i = (C_i, D_i, P_i), \qquad D_i = P_i, \qquad m = 1$$
$$\tau_1 = (18, 50, 50), \qquad \tau_2 = (10, 20, 20)$$

All times in ms. Maximum chunk duration in (c): $\Delta = 4$ ms.

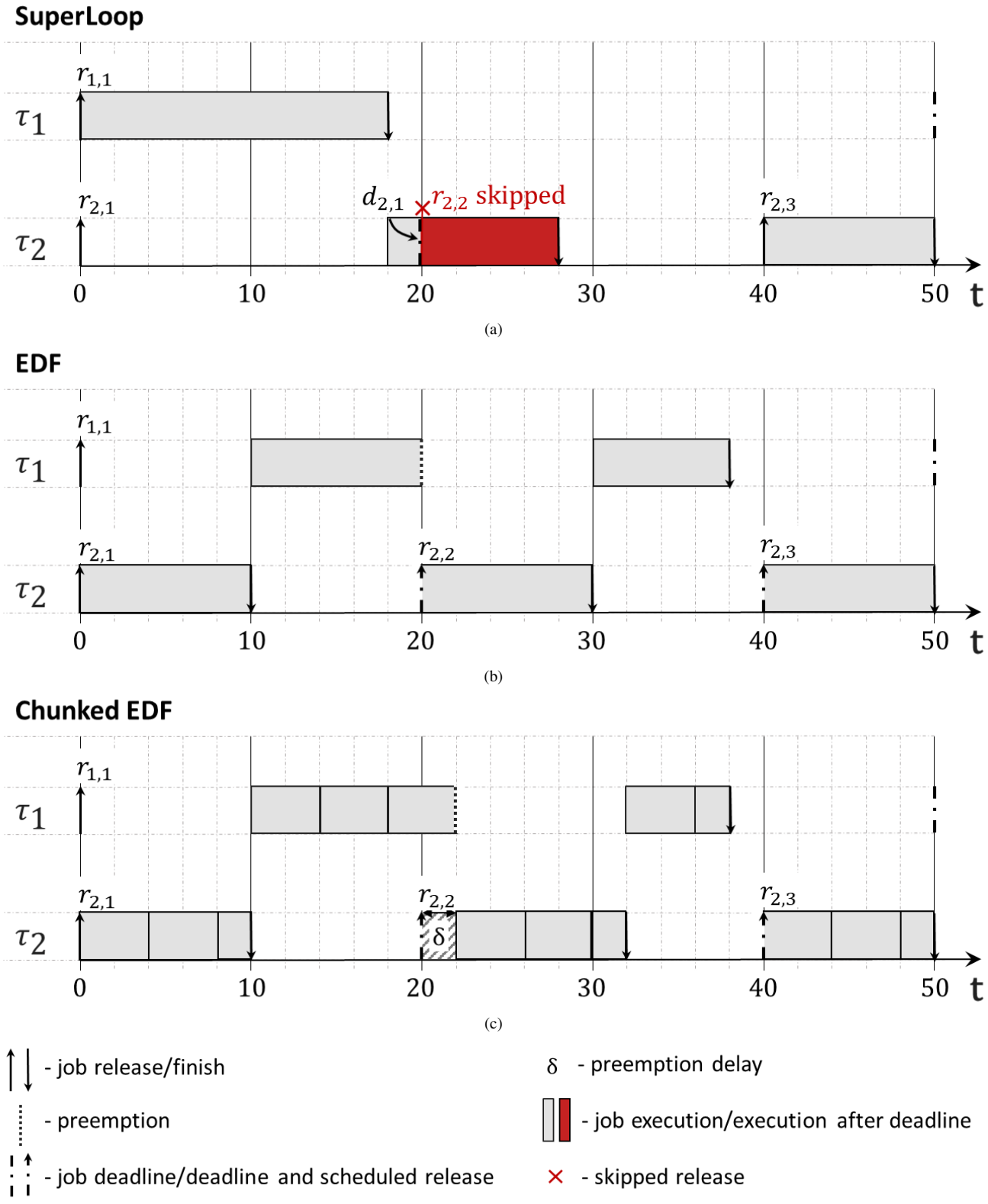



Fig. 2. Schedules for the same periodic task set under (a) SuperLoop, (b) EDF, and (c) chunked EDF.

## REFERENCES


[1] *STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx: Datasheet*, STMicroelectronics. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f767zi.pdf

[2] STMicroelectronics, "Process and environment monitoring," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/process-and-environment-monitoring.html

[3] ——, "Condition monitoring / predictive maintenance," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/condition-monitoring-predictive-maintenance.html

[4] Arm Keil, "Execution models: Single task system (Superloop)," accessed: Oct. 4, 2026. [Online]. Available: https://www.keil.com/pack/doc/mw/Graphic/html/gui_exec_model.html

[5] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[6] *DHT11 Humidity & Temperature Sensor: Technical Data Sheet*, Aosong Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[7] *Ultrasonic Ranging Module HC-SR04*, Elecfreaks. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[8] *OV2640 Datasheet*, OmniVision Technologies. [Online]. Available: https://www.uctronics.com/download/OV2640_DS.pdf

[9] *SPW2430HR5H-B: Top Port SiSonic Microphone*, Knowles Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/218/SPW2430HR5H-B-1290924.pdf

[10] A. Burmyakov and E. Bini, "Improving the efficiency of global EDF exact schedulability test," *IEEE Transactions on Computers*, pp. 1–14, 2026.